BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE APPLICATIONS AND GEO-INFORMATICS

WEEKLY PROGRESS REPORT (06/07/2026 – 12/07/2026)

WEEK 8

Project Name	OSINT Collection from Social Media Network
PROJECT DESCRIPTION :	
	Week 8 focused on two parallel workstreams. The primary workstream was the substantial enhancement of the Nexus page PDF intelligence report generator: the report now includes a full tweet-by-tweet log for every selected handle within the chosen date range, a dedicated Neo4j graph relationship section covering direct mention edges, shared hashtag co-occurrence edges, common topic nodes, and tweet-level interaction chains (reply, retweet, quote), and an expanded inference engine combining PostgreSQL engagement data with Neo4j graph topology signals into plain-English analytical findings. The secondary workstream was the creation of the complete UML diagram suite for formal project documentation: an E-R Diagram showing all six entities and their relationships, a Use Case Diagram covering 25 use cases across five actors, a Class Diagram documenting 13 classes with attributes and methods across five architectural layers, and a Table Design and Relationships diagram presenting the full PostgreSQL and Neo4j schemas in a clean black-and-white format suitable for academic submission. All Neo4j queries in the PDF report include individual try/except fallbacks ensuring graceful degradation when the graph database is offline.
GROUP MEMBER :	Rishav Pal, Sayantan Mandal, Shivam Kumar
GROUP ID :	2026G268
GROUP GUIDE :	Dr. JHUMMARWALA ABDUL
GROUP COORDINATOR :	Dr. JHUMMARWALA ABDUL
COLLEGE GUIDE NAME :	Dr. SHWETA SAHARAN
COLLEGE GUIDE No. :	9509676076
COLLEGE GUIDE EMAIL.	SHWETA.SAHARAN@GSV.AC.IN
COLLEGE NAME :	GATI SHAKTI VISHWAVIDYALAYA

WEEKLY PROGRESS REPORT (06/07/2026 – 12/07/2026)
WEEK 8 — DAILY WORK LOG

Day 1–2 | 06–07 July 2026 | PDF Report Enhancement — Full Tweet Log & Neo4j Integration

The first two days of the week completed the Nexus PDF intelligence report enhancement, adding the full tweet-by-tweet log and integrating Neo4j graph relationship data directly into the generated report.

- Audited the existing generate_report() endpoint in backend/routes/nexus.py and identified three structural gaps: only the most-liked tweet was shown per handle, Neo4j was never queried in the report pipeline, and the inference engine relied solely on PostgreSQL heuristics with no graph topology signals.
- Implemented get_neo4j_cross_handle_data(handles) helper function: four Cypher queries targeting MENTIONED_BY_USER, SHARES_HASHTAG_WITH, AUTHORED_ABOUT, and REPLY_TO|RETWEET_OF|QUOTES relationship types. Each query is wrapped in an individual try/except so a schema mismatch or partial failure does not abort the others.
- Added the full tweet log table to the per-handle PDF section: a six-column table (index, timestamp, full tweet text, likes, retweets, replies) with repeatRows=1 so the header repeats across pages, 280-character text truncation with ellipsis for readability, and an offline fallback paragraph when no tweets are available.
- Added Neo4j Graph Relationships as a new Section 3 in the PDF, with four sub-tables: Direct Mention Edges (MENTIONED_BY_USER), Shared Hashtag Edges (SHARES_HASHTAG_WITH), Common Topic Nodes (AUTHORED_ABOUT shared by 2+ handles), and Tweet-Level Interactions (REPLY_TO, RETWEET_OF, QUOTES). All four show offline fallback messages when Neo4j is unavailable.
- Extended the inference engine with five new generators: strongest graph-confirmed mention link, highest hashtag co-occurrence pair, dominant shared topic, tweet-level network density characterisation (densely vs loosely connected), and an activity summary. Added a final fallback inference when both databases are offline.
- Renumbered the existing Inferences section from Section 3 to Section 4 to accommodate the new Neo4j section. Ran python -m py_compile to confirm syntax correctness without requiring a live database connection.

Day 3 | 08 July 2026 | UML Documentation — E-R Diagram (5.2.1)

Day 3 produced the formal Entity-Relationship Diagram for the project, documenting the complete data model across both the PostgreSQL relational schema and the Neo4j graph schema.

- Designed the E-R diagram using Chen/Crow-Foot hybrid notation with six entities: USER, TWEET, HASHTAG, TOPIC, LINK, and SCRAPE_JOB. Each entity box shows all attributes with data types, using colour-coded row backgrounds to distinguish Primary Keys (blue), Foreign Keys (amber), and multi-valued array attributes (pink).
- Documented all relationships with correct cardinality: USER POSTS TWEET (1:N), TWEET MENTIONS USER (M:N via mentioned_handles[]), TWEET TAGGED_WITH HASHTAG (M:N via hashtags[]), TWEET CLASSIFIED_AS TOPIC (M:N via topics[]), TWEET CONTAINS LINK (M:N via links JSONB), USER TRIGGERS SCRAPE_JOB (1:N).
- Added three self-referential relationships on the TWEET entity shown as curved dashed arcs on the right side: REPLY_TO, RETWEET_OF, and QUOTES — each representing the optional FK columns reply_to_tweet_id, retweet_of_tweet_id, and quote_of_tweet_id.
- Included a storage annotation explaining that USER and TWEET are stored in both PostgreSQL (relational tables) and Neo4j (graph nodes), with Neo4j adding relationship edges not present in the relational schema.
- Produced the diagram as a high-resolution 180 DPI PNG using matplotlib, with complete legend showing all notation types and a cardinality reference table.

Day 4 | 09 July 2026 | UML Documentation — Use Case Diagram (5.2.2)

Day 4 produced the Use Case Diagram documenting all system interactions across five actors and twenty-five use cases.

- Defined five actors: OSINT Analyst (primary human), System Administrator (secondary human), Twitter/X Platform (external system), PostgreSQL Database (external system), and Neo4j Graph DB (external system). Human actors are represented as stick figures; system actors as labelled boxes per UML convention.
- Organised 25 use cases into four internal sections within the system boundary: Scraping & Ingestion (start/stop scraper, monitor live feed, configure handles, set tweet limit, view job history), Analytics & Exploration (view dashboard, handle intelligence, hashtag analysis, topic analysis, network graph, Nexus graph, campaigns, tweet analysis, influence chains, data explorer), Reporting & Export (export PDF, filter by date range, select handles, autocomplete search), and Configuration (apply graph search, expand node, apply influence filter, view side panel).
- Added five internal system use cases shown in a distinct green colour: Scrape Twitter/X Timelines, Parse & Enrich Tweet Data, Store to PostgreSQL, Build Neo4j Graph, and Broadcast WebSocket Events — representing the automated pipeline triggered by the scraper.
- Documented <> chains: Start Scraper includes Configure Handles, which includes Scrape Twitter, which includes Parse & Enrich, which includes Store to PostgreSQL, which includes Build Neo4j Graph. Monitor Live Feed includes Broadcast WebSocket Events.
- Documented <> relationships: Autocomplete Search extends Nexus Graph, Expand Node extends Nexus Graph, Filter by Date extends Nexus Graph, Apply Graph Search extends Network Graph, View Side Panel extends Nexus Graph.

WEEKLY PROGRESS REPORT (06/07/2026 – 12/07/2026)
WEEK 8 — DAILY WORK LOG (continued)

Day 5 | 10 July 2026 | UML Documentation — Class Diagram (5.2.3)

Day 5 produced the Class Diagram documenting all 13 classes in the Linkmap codebase with their complete attributes, methods, and UML relationships.

- Organised 13 classes into five architectural layers: Data Models (UserProfile, TweetData — Pydantic BaseModel subclasses), Scraper Layer (TweetScraper, BrowserSession, RateLimiter, Parser), Storage Layer (PostgresWriter, MemoryCache, PostgresClient), Graph ETL Layer (GraphTransformer, CypherLoader), and Infrastructure / API Layer (ScraperManager singleton, InMemoryRateLimiter, FastAPIServer).
- Each class box uses the standard UML three-compartment format: stereotype and class name in the header, attributes in the middle compartment with visibility markers (+/-) and type annotations, and methods in the lower compartment with parameter lists and return types. Primary Key and Foreign Key attributes are bold and blue.
- Documented six UML relationship types with correct notation: Composition (TweetScraper composes BrowserSession, RateLimiter, Parser), Aggregation (FastAPIServer aggregates ScraperManager and MemoryCache), Association (UserProfile authored-by relationship to TweetData via author_handle FK), Dependency / <> (TweetScraper creates UserProfile and TweetData instances), Dependency / <> (ScraperManager triggers GraphTransformer), and Dependency / <> (GraphTransformer reads PostgresClient).
- Documented the ScraperManager singleton pattern explicitly: class header shows stereotype 'backend.singleton', the _instance class attribute is listed, and get_instance() is the factory method — making the singleton pattern unambiguous in the diagram.
- Produced the diagram at 180 DPI on a 32×24 inch canvas, giving sufficient resolution for all attribute and method text to remain legible when printed at A3 size.

Day 6 | 11 July 2026 | Database Design Diagram (5.3.1)

Day 6 produced the Table Design and Relationships diagram showing the complete database schema for both the PostgreSQL relational layer and the Neo4j graph layer, formatted strictly in black and white for academic document submission.

- Designed the PostgreSQL section showing two full tables side by side: profiles (8 columns: handle PK, display_name, bio, followers_count, following_count, verified, profile_url, scraped_at) and tweets (19 columns: tweet_id PK, author_handle FK, text, timestamp, likes, retweets, replies, quote_count, is_reply, is_retweet, is_quote, reply_to_tweet_id FK*, retweet_of_tweet_id FK*, quote_of_tweet_id FK*, hashtags[], mentioned_handles[], links JSONB, topics[], scraped_at).
- Each table uses a three-column layout: column name in monospace bold, data type in italic monospace, and constraint in monospace. Row backgrounds are differentiated: medium grey for Primary Key rows, light grey for Foreign Key rows, and alternating white/near-white for normal rows.
- Drew the FK relationship connector as a precise L-shaped line (no diagonal) running from the left edge of the tweets table at the author_handle row, horizontally through the gap between tables, vertically to the handle row height, then horizontally to the right edge of the profiles table — with an arrowhead pointing to the referenced PK and 1/N cardinality labels.
- Drew three self-referential FK connectors on the right side of the tweets table as U-shaped lines (staggered at 0.4, 0.8, and 1.2 inch extensions) connecting reply_to_tweet_id, retweet_of_tweet_id, and quote_of_tweet_id back to tweet_id. Each arm is labelled REPLY_TO, RETWEET_OF, QUOTES.
- Added the Neo4j section with three node definition boxes (User, Tweet, Topic) showing property names and types, followed by a seven-row relationship edge table listing all Cypher relationship patterns with semantic descriptions.

- Ensured strict separation of all visual elements: 1.3-inch gap between the two tables for the FK connector, self-ref arms ending before the Neo4j section boundary, no text overlapping any connection line. Output saved at 200 DPI on a 26×21 inch canvas.

Day 7 | 12 July 2026 | Sequence Diagram, Documentation Review & Final Wrap-Up

The final day of the project completed the Sequence Diagram, conducted a full review of all diagrams produced during the week, and closed out the documentation package for final submission.

- Designed the Sequence Diagram (5.2.4) covering the primary end-to-end flow: Analyst requests scrape via the frontend, the SPA sends a POST to /api/scraper/start, the FastAPI rate limiter validates the request, ScraperManager creates a job and spawns a worker thread, the worker launches a BrowserSession via Playwright, authenticates using cookie storage, scrapes the target Twitter/X timeline with human-delay scrolling, parses and enriches each tweet via the Parser, writes to PostgreSQL in batches via PostgresWriter, and broadcasts progress events to the frontend via WebSocket.
- Documented the post-scrape graph pipeline sequence as a secondary flow: ScraperManager triggers run_graph_pipeline(), GraphTransformer reads all profiles and tweets from PostgreSQL, writes nodes.csv and edges.csv, CypherLoader executes LOAD CSV Cypher statements against Neo4j, and ScraperManager broadcasts a GRAPH_READY WebSocket event to all connected frontend clients.
- Reviewed all four diagrams produced during the week against the project codebase to confirm accuracy: verified entity names match Pydantic model field names, class attribute lists match actual class definitions, use case names match actual route/function names, and table columns match the PostgreSQL schema defined in storage/schema.py.
- Compiled the final documentation package: consolidated the PDF report enhancements and all four UML diagrams (E-R, Use Case, Class, and Database Design) into a single submission-ready set, cross-checked against the live codebase for consistency.
- Confirmed all diagrams meet the submission requirements: black and white formatting, bold text for legibility, connection lines terminating at box edges without overlap, and sufficient resolution (180-200 DPI) for A3/A4 print quality.

WEEKLY PROGRESS REPORT (06/07/2026 – 12/07/2026)
WEEK 8 — MEMBER CONTRIBUTIONS

Rishav Pal — Data Engineer

- Led the PDF report enhancement: implemented get_neo4j_cross_handle_data() with all four Cypher queries (MENTIONED_BY_USER, SHARES_HASHTAG_WITH, AUTHORED_ABOUT, REPLY_TO|RETWEET_OF|QUOTES), each in individual try/except blocks for offline-safe operation.
- Integrated the neo4j_data call into generate_report() with a two-line insertion before the cross-handle relationship computation block. Confirmed correct placement and no disruption to existing endpoint behaviour.
- Designed and verified the E-R Diagram (5.2.1): confirmed entity names, attribute names, data types, and relationship cardinalities against the actual Pydantic schema definitions in storage/schema.py and the PostgreSQL table definitions.
- Reviewed the Database Design Diagram (5.3.1) for correctness against the live schema: confirmed all 8 profiles columns and all 19 tweets columns are present with correct data types and constraint annotations.
- Ran static verification on all modified Python files (py_compile), confirmed no syntax errors introduced by the PDF enhancement changes.
- Compiled the final project documentation package, consolidating all diagram outputs and PDF report changes into a single submission-ready set for handover.

Sayantan Mandal — OSINT Analyst

- Designed and produced the Use Case Diagram (5.2.2): defined all five actors, 25 use cases in four system boundary sections, five internal system use cases, the <> pipeline chain, and five <> relationships. Confirmed use case names match actual API route paths and frontend function names.
- Added the full tweet log table to the PDF report Section 1: implemented the six-column tweet table with repeatRows=1, 280-character text truncation, 7pt row font for density, and offline/empty fallback paragraph.
- Added the Neo4j Section 3 to the PDF: designed and implemented four sub-tables (Direct Mention Edges, Shared Hashtag Edges, Common Topic Nodes, Tweet-Level Interactions) with human-readable relationship type labels and offline fallback messages for each.
- Designed the Sequence Diagram (5.2.4): mapped the complete interaction sequence from Analyst trigger through scrape pipeline to WebSocket broadcast, identifying all synchronous and asynchronous message exchanges between lifelines.
- Conducted accuracy review of the Class Diagram against the actual codebase: verified TweetScraper, PostgresWriter, ScraperManager, GraphTransformer, and CypherLoader method signatures.
- Carried out a final walkthrough of all seven dashboard pages and the PDF report generator to confirm every feature delivered during the project is working end-to-end ahead of submission.

Shivam Kumar — Graph Analyst

- Designed and produced the Class Diagram (5.2.3): documented all 13 classes with complete attribute lists (visibility, name, type), method signatures (visibility, name, params, return type), and six UML relationship types with correct notation.
- Designed and produced the Table Design & Relationships Diagram (5.3.1) in strict black and white: implemented the L-shaped FK connector with precise edge-to-edge line termination, three staggered U-shaped self-referential FK connectors, and the Neo4j node and relationship edge sections.
- Extended the PDF inference engine: implemented five new inference generators using neo4j_data output (strongest mention, hashtag co-occurrence, dominant topic, network density, activity summary) plus the offline fallback inference

for empty results.

- Ensured all diagram line connectors terminate exactly at box edges with no overlap: calculated precise L-connector midpoints, staggered self-ref arm extensions, and maintained separation between sections.
- Confirmed diagram resolution requirements: all PNG outputs at 180-200 DPI on large canvases (24-32 inches) ensuring A3 print legibility for academic submission.
- Performed a final review of the Neo4j graph schema and all Cypher queries used across the backend and PDF report to confirm consistency ahead of project submission.

WEEKLY PROGRESS REPORT (06/07/2026 – 12/07/2026)
WEEK 8 — TECHNICAL SUMMARY

Feature Overview — PDF Report Enhancement & UML Documentation Suite

Week 8 delivered two parallel bodies of work. On the product side, the Nexus PDF intelligence report was transformed from a summary-level document into a fully detailed artefact covering all tweets in the selected period, complete Neo4j graph relationship evidence, and enriched inferences drawing on both relational and graph data sources. On the documentation side, four formal UML diagrams were produced covering the entity-relationship model, use cases, class structure, and database schema — all verified for accuracy against the live codebase.

Deliverable	Technology / Method	Key Detail
get_neo4j_cross_handle_data()	Python + Neo4j Cypher	4 queries: MENTIONED_BY_USER, SHARES_HASHTAG_WITH, AUTHORED_ABOUT, REPLY_TO|RETWEET_OF|QUOTES. Each individually try/excepted. Offline-safe.
Full tweet log (PDF Section 1)	ReportLab Table	6-column table, repeatRows=1, 280-char truncation, 7pt font, alternating shading, empty-state fallback.
Neo4j relationships (PDF Section 3)	ReportLab + Cypher	4 sub-tables: mention edges, hashtag edges, topic nodes, tweet interactions. Offline fallback per table.
Expanded inference engine	Python heuristics + graph	5 new inferences: mention strength, hashtag co-occurrence, dominant topic, network density, activity summary. Offline fallback when both DBs unavailable.
E-R Diagram (5.2.1)	matplotlib (180 DPI PNG)	6 entities, 6 relationships, 3 self-ref, colour-coded PK/FK/array rows, cardinality notation.
Use Case Diagram (5.2.2)	matplotlib (180 DPI PNG)	5 actors, 25 use cases, 4 system sections, <> chain, 5 <> relationships.
Class Diagram (5.2.3)	matplotlib (180 DPI PNG)	13 classes, 5 layers, full attributes + methods, 6 UML relationship types.
DB Design Diagram (5.3.1)	matplotlib (200 DPI PNG)	B&W;, profiles + tweets full schema, L-shaped FK connector, 3 self-ref U-connectors, Neo4j node + edge section.
Sequence Diagram (5.2.4)	UML Sequence notation	Scrape pipeline + graph pipeline lifelines, sync and async messages, WebSocket broadcast.

Week 8 Deliverables Summary

Deliverable	File	Status
get_neo4j_cross_handle_data() + 4 Cypher queries	backend/routes/nexus.py	Complete
Full tweet log table (PDF Section 1)	backend/routes/nexus.py	Complete
Neo4j Mention Edges table (PDF Section 3)	backend/routes/nexus.py	Complete
Neo4j Shared Hashtag Edges table (PDF Section 3)	backend/routes/nexus.py	Complete
Neo4j Common Topic Nodes table (PDF Section 3)	backend/routes/nexus.py	Complete
Neo4j Tweet Interactions table (PDF Section 3)	backend/routes/nexus.py	Complete
5 neo4j-derived inferences + offline fallback	backend/routes/nexus.py	Complete
Static verification (py_compile, no DB required)	terminal	Passed
E-R Diagram — 6 entities, 6 relationships	5_2_1_ER_Diagram.png	Complete
Use Case Diagram — 25 use cases, 5 actors	5_2_2_Use_Case_Diagram.png	Complete
Class Diagram — 13 classes, 5 layers	5_2_3_Class_Diagram.png	Complete
Table Design & Relationships — B&W schema	5_3_1_DB_Design.png	Complete

Sequence Diagram — scrape + graph pipeline	5_2_4_Sequence_Diagram	Complete

PDF Report — Section Structure Comparison (Week 5 → Week 8)

Section	Original (Week 5)	Enhanced (Week 8)
Section 1 Handle Profiles	Stats table, hashtag/mention tables, topics, one most-liked tweet only	Stats table, hashtag/mention tables, topics, most-liked tweet + FULL tweet log for all tweets in selected period
Section 2 Cross-Handle (PostgreSQL)	Shared hashtags, cross mentions, shared links	Unchanged — same three PostgreSQL-derived tables as original
Section 3 (NEW Week 8)	Did not exist	Neo4j Graph Relationships: Direct Mention Edges, Shared Hashtag Edges, Common Topic Nodes, Tweet-Level Interactions
Section 4 Inferences	5 PostgreSQL heuristics: most active, most influential, coordination, cross-mentions, common topics	Original 5 + 5 Neo4j inferences: graph mention, co-occurrence, dominant topic, network density, activity summary + offline fallback

---